

Análisis Léxico y Símbolos

Fundamentos Formales

DRA. MELISSA BRIGITTHE HINOJOSA CABELLO

Universidad Nacional Autónoma de México

Facultad de Ingeniería

22 de Septiembre de 2026





Tarea 6: Fundamentos formales de análisis léxico y símbolos

→ Investigar acerca de:

- Lenguajes regulares
- Expresiones regulares
- Autómatas finitos deterministas
- Autómatas finitos no deterministas

→ Elaborar un reporte con las definiciones de los puntos anteriores.



→ Responder brevemente (con sus palabras) las siguientes preguntas:

- ¿Cuál es la relación entre expresiones regulares y autómatas finitos deterministas?
- ¿Por qué el análisis léxico puede modelarse con un AFD?
- ¿Qué ocurriría si un token requiriera memoria no acotada para reconocerse?

→ Entrega: → Entrega: **Martes 22/Septiembre/2026** (antes de la clase).

- Enviar reporte (digital) por correo.
- Discusión en clase.



¿Qué es un lenguaje regular?

Lenguaje regular

Es un conjunto de cadenas sobre un alfabeto finito que puede describirse mediante una expresión regular, reconocerse mediante un autómata finito o generarse mediante una gramática regular.

→ Es decir, que puede reconocerse utilizando memoria finita.

Los tokens de un lenguaje de programación forman lenguajes regulares. Ejemplos:

- ▶ Identificadores: `identificador` → `letra (letra | dígito)*`
- ▶ Números enteros: `números` → `dígito+`
- ▶ Palabras reservadas: `if` | `while` | `return`



¿Todo lenguaje es regular?

→ Los lenguajes regulares **NO** necesitan memoria ilimitada.

Si un lenguaje requiere contar cantidades arbitrarias, emparejar símbolos o manejar estructuras anidadas, entonces ya no es regular.

Dado el lenguaje: $L = \{a^n b^n | n \geq 0\}$

¿Cuáles son las cadenas válidas que produce? ¿Es regular? ¿Por qué?



¿Qué es una expresión regular?

Expresión regular

Es una notación formal utilizada para describir conjuntos de cadenas que pertenecen a un lenguaje regular permitiendo especificar los patrones que deberán cumplir las cadenas para considerarse válidas.

→ Es una representación algebraica utilizada para describir conjuntos de cadenas que pertenecen a un lenguaje regular.



Una expresión regular se construye a partir de símbolos de un alfabeto finito y de tres operaciones fundamentales:

- Unión ($A \mid B$) $\rightarrow$ ¿Qué expresa esta operación?

Permite expresar que una cadena puede pertenecer a uno u otro conjunto.

- Concatenación (AB) $\rightarrow$ ¿Qué permite esta operación?

Permite construir secuencias: un patrón debe ir seguido de otro.

- Cerradura de Kleene (A^*) $\rightarrow$ ¿Qué permite esta operación?

Permite repetir un patrón cero o más veces.

$\rightarrow$ Estas operaciones permiten combinar patrones simples para formar descripciones más complejas, siendo suficientes para describir todos los lenguajes regulares.



¿Qué es un autómata finito determinista?

Autómatas finitos deterministas (AFD)

Es un modelo matemático utilizado para reconocer lenguajes regulares. Se denomina determinista porque, para cada estado y cada símbolo del alfabeto, existe exactamente una transición posible.

Características:

- ▶ No tiene memoria ilimitada.
- ▶ No tiene pila.
- ▶ No puede contar arbitrariamente.
- ▶ Solo puede cambiar de estado.

→ **Finito:** El número de estados es limitado.

→ **Determinista:** Para cada símbolo leído, solo hay un camino posible.



¿Cómo funciona un autómata finito determinista?

El funcionamiento de un AFD consiste en:

- ▶ Leer la cadena símbolo por símbolo.
- ▶ Cambiar de estado según la función de transición.
- ▶ Si al finalizar la lectura la máquina se encuentra en un estado de aceptación, la cadena pertenece al lenguaje.
- ▶ En caso contrario, la cadena es rechazada.

Estados del autómata:

- q_0 , estado inicial
- q_1 , estado de aceptación (identificador válido)

Transiciones:

- Desde q_0 : Si lee letra $\rightarrow$ Va a q_1 .
- Desde q_1 : Si lee letra o dígito $\rightarrow$ Permanece en q_1



Ejemplo:

Lenguaje $\rightarrow$ letra (letra | dígito)*

Cadena $\rightarrow$ var1

► Primer símbolo: *v*

- Estado actual: *q0*
- ¿Es letra? *sí*

- Transición: $q0 \xrightarrow{\text{letra}} q1$
- Nuevo estado actual: *q1*

► Segundo símbolo: *a*

- Estado actual: *q1*
- ¿Es letra o dígito? *sí, letra*

- Transición: $q1 \xrightarrow{\text{letra}} q1$
- Nuevo estado actual: *q1*



► Tercer símbolo: r

- Estado actual: $q1$
- ¿Es letra o dígito? *sí, letra*

- Transición: $q1 \xrightarrow{\text{letra}} q1$
- Nuevo estado actual: $q1$

► Cuarto símbolo: 1

- Estado actual: $q1$
- ¿Es letra o dígito? *sí, dígito*

- Transición: $q1 \xrightarrow{\text{dígito}} q1$
- Nuevo estado actual: $q1$

► Fin de la cadena (ya no hay más símbolos)

- Estado actual: $q1$
- ¿Es estado de aceptación? *sí*

- Cadena aceptada.



Que **SÍ** hace el autómata:

- Leer símbolos.
- Cambiar de estado.
- Recordar en qué estado está.
- Decidir al final.

Que **NO** hace el autómata:

- Almacenar la cadena completa.
- Contar letras.
- Comparar cantidades.

Entonces, ¿en qué consiste un autómata finito determinista?

Consiste en un conjunto finito de estados, un alfabeto de entrada, una función de transición que determina de manera única el siguiente estado para cada símbolo leído, un estado inicial y un conjunto de estados de aceptación.



¿Qué es un autómata finito no determinista?

Autómatas finitos no deterministas (AFN)

Al igual que el AFD, es un modelo matemático utilizado para reconocer lenguajes regulares. La diferencia radica en que para un mismo estado y un mismo símbolo pueden existir varias transiciones posibles, o incluso transiciones que no consumen un símbolo.

- **Finito:** El número de estados es limitado.
- **No Determinista:** Ante una entrada, se puede “elegir” entre distintos caminos posibles.



Entonces, ¿en qué consiste un autómata finito no determinista?

Consiste en un conjunto finito de estados, un alfabeto de entrada, una función de transición, un estado inicial y uno o más estados de aceptación.

→ Una cadena es aceptada si existe al menos un camino que termine en un estado de aceptación.



Ejemplo: Lenguaje $\rightarrow ab \mid ac$, Cadena $\rightarrow ab$

► Primer símbolo: a

- Estado actual: q_0 .
- ¿Existe transición con 'a'? *sí, existen dos posibles transiciones.*

- Transiciones: $q_0 \xrightarrow{a} q_1$, $q_0 \xrightarrow{a} q_2$
- El autómata puede continuar por ambos caminos.

► Segundo símbolo: b

• Primer camino (q_1)

- Estado actual: q_1 .
- ¿Existe transición con 'b'? *sí.*
- Transición: $q_1 \xrightarrow{b} q_f$
- Nuevo estado actual: q_f .
- ¿Es estado de aceptación? *sí.*

• Segundo camino (q_2)

- Estado actual: q_2 .
- ¿Existe transición con 'b'? *no.*
- Este camino se rechaza.



Ejemplo: Lenguaje $\rightarrow ab \mid ac$, Cadena $\rightarrow ab$

- ▶ Fin de la cadena (ya no hay más símbolos)
 - Existe al menos un camino que llegó a estado de aceptación.
 - Cadena aceptada.

Ejemplo: Lenguaje $\rightarrow ab \mid ac$, Cadena $\rightarrow ad$

- ▶ Primer símbolo: a
 - Estado actual: q_0
 - ¿Existe transición con 'a'? *sí, existen dos posibles transiciones.*
 - Transiciones: $q_0 \xrightarrow{a} q_1$, $q_0 \xrightarrow{a} q_2$
 - El autómata puede continuar por ambos caminos.



Ejemplo: Lenguaje $\rightarrow ab \mid ac$, Cadena $\rightarrow ad$

► Segundo símbolo: d

- Primer camino ($q1$)

- Estado actual: $q1$.
- ¿Existe transición con 'd'? *no*.
- Este camino se rechaza.

- Segundo camino ($q2$)

- Estado actual: $q2$.
- ¿Existe transición con 'd'? *no*.
- Este camino se rechaza.

► Fin de la cadena (ya no hay más símbolos)

- Ningún camino llegó a estado de aceptación.
- Cadena rechazada.



Recordatorio:

AFD → Solo hay un camino posible.

vs.

AFN → Hay más de un camino posible.

→ Una cadena es aceptada si existe al menos un camino que termine en un estado de aceptación.